PYTHON CONTROL FLOW & OPERATORS

Simple examples of while, do-while alternative, match-case, break, continue, pass and important operators

Beginner-Friendly Notes

Prepared for easy learning, classroom explanation and practice.

1. while Loop

A **while** loop repeats a block of code as long as its condition remains **True**.

```
number = 1

while number <= 5:
    print(number)
    number = number + 1
```

OUTPUT

```
1
2
3
4
5
```

Real-world example - password checking

```
password = ""

while password != "python123":
    password = input("Enter your password: ")

print("Login successful!")
```

How it works: The loop keeps asking for the password. It stops only when the user enters **python123**.

2. Do-While Alternative

Python does not have a direct **do-while** statement. We can create the same behaviour with **while True** and **break**. The loop body executes at least once.

```
while True:
    number = int(input("Enter a positive number: "))

    if number > 0:
        break

    print("Please enter a positive number.")

print("You entered:", number)
```

Important: The input statement runs before the break condition is checked, so the code executes at least once.

3. Switch Alternative: match-case

Python 3.10 and later provide **match-case**. It is similar to a switch statement in Java, C# and other languages.

```
day = int(input("Enter a day number from 1 to 3: "))

match day:
    case 1:
        print("Monday")
    case 2:
        print("Tuesday")
    case 3:
        print("Wednesday")
    case _:
        print("Invalid day number")
```

case _ works like the **default** option in a switch statement.

Simple calculator using match-case

```
number1 = int(input("Enter first number: "))
number2 = int(input("Enter second number: "))
operator = input("Enter +, -, * or /: ")

match operator:
    case "+":
        print("Result:", number1 + number2)
    case "-":
        print("Result:", number1 - number2)
    case "*":
        print("Result:", number1 * number2)
    case "/":
        if number2 != 0:
            print("Result:", number1 / number2)
        else:
            print("Cannot divide by zero")
    case _:
        print("Invalid operator")
```

4. break

break immediately stops the complete loop.

```
number = 1

while number <= 10:
    if number == 5:
        break

    print(number)
    number += 1

print("Loop stopped")
```

OUTPUT

```
1
2
3
4
Loop stopped
```

5. continue

continue skips only the current iteration and moves to the next iteration.

```
number = 0

while number < 5:
    number += 1

    if number == 3:
        continue

    print(number)
```

OUTPUT

```
1
2
4
5
```

The number **3** is skipped, but the loop continues.

6. pass

pass does nothing. It is used as a placeholder when code will be added later.

```
age = 20

if age >= 18:
    pass

print("Program completed")
```

```
def payment():
    pass

print("Payment function will be developed later")
```

Keyword	Meaning
break	Stops the complete loop
continue	Skips the current iteration
pass	Does nothing; used as a placeholder

7. Arithmetic Operators

```
a = 10
b = 3

print("Addition:", a + b)
print("Subtraction:", a - b)
print("Multiplication:", a * b)
print("Division:", a / b)
print("Floor division:", a // b)
print("Remainder:", a % b)
print("Power:", a ** b)
```

OUTPUT

```
Addition: 13
Subtraction: 7
Multiplication: 30
Division: 3.3333333333333335
Floor division: 3
Remainder: 1
Power: 1000
```

Operator	Purpose	Example result
+	Addition	10 + 3 = 13
-	Subtraction	10 - 3 = 7
*	Multiplication	10 * 3 = 30
/	Normal division	10 / 3 = 3.33
//	Division without decimal	10 // 3 = 3
%	Remainder	10 % 3 = 1
**	Power	10 ** 3 = 1000

8. Comparison Operators

Comparison operators compare values and return either **True** or **False**.

```
a = 10
b = 5

print(a == b)   # False
print(a != b)   # True
print(a > b)    # True
print(a < b)    # False
print(a >= b)   # True
print(a <= b)   # False
```

Comparison Operator - Real-world Example

```
age = int(input("Enter your age: "))

if age >= 18:
    print("You are eligible to vote.")
else:
    print("You are not eligible to vote.")
```

9. Logical Operators

and - Both conditions must be True.

```
age = 25
has_id = True

if age >= 18 and has_id:
    print("Entry allowed")
```

or - At least one condition must be True.

```
day = "Sunday"

if day == "Saturday" or day == "Sunday":
    print("It is a weekend")
```

not - Reverses the result.

```
is_raining = False

if not is_raining:
    print("You do not need an umbrella")
```

10. Assignment Operators

```
number = 10

number += 5
print(number)       # 15

number -= 3
print(number)       # 12

number *= 2
print(number)       # 24

number /= 4
print(number)       # 6.0
```

Shortcut: `number += 5` means `number = number + 5`.

11. Membership Operators

Membership operators check whether a value is present in a collection such as a list, tuple, set or string.

```
fruits = ["Apple", "Mango", "Banana"]

print("Mango" in fruits)          # True
print("Orange" not in fruits)     # True
```

Combined Simple Program

This menu-driven program combines **while**, **match-case**, arithmetic and comparison operators, **break** and **continue**.

```
while True:
    print("\n--- MENU ---")
    print("1. Addition")
    print("2. Check Even or Odd")
    print("3. Exit")

    choice = int(input("Enter your choice: "))

    match choice:
        case 1:
            a = int(input("Enter first number: "))
            b = int(input("Enter second number: "))
            print("Answer:", a + b)

        case 2:
            number = int(input("Enter a number: "))

            if number % 2 == 0:
                print("Even number")
            else:
                print("Odd number")

        case 3:
            print("Program closed")
            break

        case _:
            print("Invalid choice")
            continue
```

Practice: Add two more menu options: subtraction and checking whether a number is positive, negative or zero.